Medi Vault - Product Requirements Document (PRD)

Emon Kalyan and Nikhil

October 2026

1. Problem Statement

Hospitals and clinics create many documents for each patient: prescriptions, lab reports, discharge summaries and scanned papers. When these are kept in paper files or in unorganised folders, finding one report is slow, files get lost, and nobody can tell who looked at a sensitive record. Medical data is private, so a digital replacement must also control who can see what and keep a record of access.

Medi Vault is a small web application that lets authorised hospital staff register patients, store their documents privately, find them quickly, and review an audit trail of every important action. It is inspired by the publicly described NeuralBits Medi Vault product category; this project is an independent learning build and does not use NeuralBits' code.

2. Target Users

User (role)	What they do in the app
Records clerk	Registers patients, uploads documents, searches and opens records
Doctor	Searches patients and opens their documents (read-only)
Auditor / security reviewer	Reviews the audit log of who did what and when (no access to patient data)

3. Goals

1. Give staff one place to register patients and attach documents to them.
2. Make records easy to find by patient name or medical record number (MRN).
3. Allow only logged-in users with the right role to do each action, checked on the server.
4. Record every important action, including failed logins and denied attempts.
5. Work end to end with **synthetic (fake) data only**, and start with one command.

4. Features

Feature	Priority	Status
Login / logout with hashed passwords (PBKDF2-SHA256)	Must-have	**Built**
Role-based access: clerk, doctor, auditor (checked server-side)	Must-have	**Built**
Register patient with validation and unique MRN	Must-have	**Built**
Search patients by name or MRN	Must-have	**Built**
Patient page with document list	Must-have	**Built**
Secure upload: PDF/PNG/JPG only, max 5 MB, file contents verified, random stored name	Must-have	**Built**
Protected document viewing (login and role required, access logged)	Must-have	**Built**
Audit log: logins, failed logins, searches, views, uploads, rejected uploads, denied attempts	Must-have	**Built**
Seed data (3 fake patients, 1 fake report) and one-command run	Must-have	**Built**

CSRF protection on all forms, secrets in `.env`, security headers	Must-have	**Built**
Encounters (OPD / IPD visits)	Nice-to-have	Not built
OCR with human verification	Nice-to-have	Not built
Multiple hospitals (data isolation per organisation)	Nice-to-have	Not built
Document versions, virus scanning, multi-factor login, login lockout	Nice-to-have	Not built
Admin screen to manage users; edit and delete patients	Nice-to-have	Not built
FHIR / ABDM integration	Nice-to-have	Not built

5. User Flow

1. A user opens the app and logs in. The server creates a session and records the login.
2. **Clerk:** Patients page -> search or **Register patient** -> open the patient -> **Upload** a document -> **Open** it.
3. **Doctor:** Patients page -> search -> open the patient -> **Open** a document. No register or upload forms are shown.
4. **Auditor:** is taken directly to the **Audit log**. Opening patient pages is blocked and the attempt is logged.
5. Any user can log out. Every step above writes an audit entry.

6. Tech Stack

Part	Choice
Language / framework	Python 3.9+, Flask
Pages	Server-rendered HTML (Jinja templates) with a little CSS
Database	SQLite (a single file, `medivault.db`)
File storage	Private local folder `private_uploads/` (not served directly)
Password hashing	Werkzeug PBKDF2-SHA256
Setup	`run.sh` creates a virtual environment, installs Flask, creates `.env` with a random secret, starts the app

7. Database / Data Model

Table	Main fields
`users`	id, username (unique), password hash, role
`patients`	id, mrn (unique), name, date of birth, created by, created at
`documents`	id, patient id (foreign key), title, type, stored name (random), original name, size, SHA-256 checksum, uploaded by, created at
`audit`	id, time, user, action, detail, outcome (ok / failed / denied / rejected)

A patient has many documents. Queries use parameters (not string joining) to prevent SQL injection.

8. Security and Privacy Considerations

- **Passwords** are stored only as hashes. Demo passwords are fake and come from `.env`.
- **Authentication and roles:** every protected route checks login and the role's permission on the server. Denied attempts get a 403 page and an audit entry.
- **Uploads:** only PDF, PNG and JPG; maximum 5 MB; the first bytes of the file must match the claimed type; files are saved under a random name outside any public folder and can only be opened through a protected route.
- **CSRF tokens** protect every form; session cookies are `HttpOnly` and `SameSite=Lax`; responses use `nosniff` and `no-store` headers.
- **Secrets** live in `.env`, which is excluded from Git; `.env.example` shows the format.

- **Audit trail** records who did what and when, including failures.
- **Privacy:** the demo uses synthetic data only. Real patient data must never be used without a proper security and legal review (for India, including applicable data-protection and healthcare rules).

9. Scope and Limitations

This is a learning prototype, not a production hospital system. It has a single hospital, no encounters, no OCR, no virus scanning, no login lockout or multi-factor login, no HTTPS, no user-management screen, and uses SQLite and local files instead of PostgreSQL and cloud storage. Lists are limited to 100 rows. It has been tested manually and with a scripted smoke test, not with a full automated test suite. It makes no claim of legal compliance.

10. 4-Week Timeline

Week	Work done
1	Studied the Medi Vault concept and requirements, compared technology options, listed roles and workflows
2	Designed the data model (users, patients, documents, audit) and the permission model
3	Built the core: login, patients, search, document upload and viewing
4	Added security hardening (hashing, role checks, upload validation, CSRF), audit log, seed data, one-command setup, testing, PRD, README and demo video

11. Success Metrics

- A complete demo (login, register, upload, search, view, switch roles, audit log) runs in under 5 minutes.
- 100% of protected pages redirect to login or return 403 for the wrong role.
- Invalid or disguised files are rejected and logged.
- Every login, upload, document view and denied attempt appears in the audit log.
- The app starts with one command on a fresh machine.

12. Future Improvements

1. **Safety:** wrong-patient warning (compare patient details inside a document with the selected patient), duplicate-patient review.
2. **Trust:** tamper-evident audit log using a hash chain, viewer watermark with user name and time, break-glass emergency access.
3. **Access:** multiple hospitals with strict data isolation, multi-factor login, login lockout, admin screen for users and roles.
4. **Documents:** encounters (OPD / IPD), OCR with a human verification queue, versions, virus scanning, better search.
5. **Platform:** PostgreSQL and private cloud object storage, HTTPS, automated tests, backups.
6. **Interoperability:** HL7 FHIR resources and ABDM / ABHA support after proper review.